

CS-320 Fall 2026 Section A/C Project 1

In this project, you will construct an assembler for a simplified version of LEGv8 ISA, fondly named FOOTv8. The FOOTv8 ISA is very RISC-minded, almost a minimal ISA that is very simple, but not very powerful. You will be provided man readable assembly as input, and your program will need to translate each instruction its binary format.

Project Rules

1. All students must work alone. Cooperation is not allowed.
2. Sharing of code between students is considered cheating and will result in a zero grade and an Academic Honesty violation. Source Code will be checked with tools designed to uncover code which is similar even after minor modifications in things like variable names, comments, white space, etc.
3. All project submissions must be written in C code. C++ is not allowed.
4. Your code will be graded on remote.cs.binghamton.edu machines. Please test on either one of these remote machines or the machines in EB-G7, EB-Q22, or EB-N01, which all have the same software configuration.

The FOOTv8 ISA

- The instructions supported in FOOTv8 are described in table 1 on the following page
- There are 32 32-bit integer registers, named X0-X31. (This is different from LEGv8, where these are 64-bit registers.)
- All arithmetic operations (ADD, SUB, MUL, and DIV) are integer operations that take two 32 bit two's complement integer operands and produce a single 32 bit two's complement integer result. Division is integer division that rounds towards zero, like C integer division.
- The following are features of the man-readable assembly format

Assembly	Semantics	Binary Format			
		Op	B1	B2	B3
NOP	No-operation	0x00	0x00	0x00	0x00
ADD Rd,Rn,Rm	$R[Rd] = R[Rn] + R[Rm]$	0x02	Rd	Rn	Rm
ADDI Rd,Rn,#Imm	$R[Rd] = R[Rn] + Imm$	0x03	Rd	Rn	Imm
SUB Rd,Rn,Rm	$R[Rd] = R[Rn] - R[Rm]$	0x04	Rd	Rn	Rm
SUBI Rd,Rn,#Imm	$R[Rd] = R[Rn] - Imm$	0x05	Rd	Rn	Imm
MUL Rd,Rn,Rm	$R[Rd] = R[Rn] \times R[Rm]$	0x06	Rd	Rn	Rm
MULI Rd,Rn,#Imm	$R[Rd] = R[Rn] \times Imm$	0x07	Rd	Rn	Imm
DIV Rd,Rn,Rm	$R[Rd] = R[Rn] / R[Rm]$	0x08	Rd	Rn	Rm
DIVI Rd,Rn,#Imm	$R[Rd] = R[Rn] / Imm$	0x09	Rd	Rn	Imm
LD Rd,[Rb,#Imm]	$R[Rd] = Mem[R[Rb] + Imm]$	0x0B	Rd	Rb	Imm
ST Rt,[Rb,#Imm]	$Mem[R[Rb] + Imm] = R[Rt]$	0x0D	Rt	Rb	Imm
EXIT	End of Program	0x0E	0x00	0x00	0x00
MOV Rd,#Imm	$R[Rd] = Imm$	0x11	Rd	Imm	
B #Imm	$PC = PC + (4 \times Imm)$	0x13	0x00	Imm	
CBZ Rt,#Imm	If $Rt == 0$ $PC = PC + (4 \times Imm)$	0x15	Rt	Imm	
CBNZ Rt,#Imm	If $Rt \neq 0$ $PC = PC + (4 \times Imm)$	0x17	Rt	Imm	
CBNEG Rt,#Imm	If $Rt < 0$ $PC = PC + (4 \times Imm)$	0x19	Rt	Imm	

Table 1: FOOTv8 Instructions

- The op-code mnemonic must be the first non-white space token on a line. Mnemonics are case-insensitive. They may be in upper case, lower case, or mixed case.
- Operand fields must appear on the same line as the op-code. Operand fields can be separated by white space, commas, or (in the case of loads and stores) square brackets, similar to LEGv8 Assembler. The order of the fields are typically destination first, followed by operand 1 and operand 2 if required.
- The register references in table 1 (Rd, Rn, Rm, Rt) are all references to the integer registers X0 - X31.
- Immediate values are always preceded by a pound sign (#). The value itself consists of any value that can be read by the C scanf format %i, including decimal values, optionally preceded by a + or -, or a hexadecimal value preceded by 0x.
- There are no labels or label references in FOOTv8 assembler. All branch targets must be specified as an immediate numeric value that represents the number of instructions from the branch instruction.
- Comments start with // and end at the end of the assembler line.

- Empty or comment-only lines are allowed, but ignored by the assembler.
- The following are features of the binary machine code format
 - All instructions are four bytes wide. The first byte contains the binary op-code. The remaining three bytes contain the operands of the instruction.
 - The format of the operands depends on the first hex digit of the op-code. If the first digit is 0x0, then the operands are in three one-byte fields. If the first digit is 0x1, there is one one-byte field followed by a two-byte immediate value.
 - Interpretation of the fields is defined in table 1 on the preceding page. Register references consist of the numeric register index. For example, 0x0A refers to register X10.
 - The last bit of the binary op-code indicates whether there is an immediate value in the instruction or not. Odd op-codes have immediate values; even op-codes do not. The immediate value is always the last field.
 - All immediate values (*Imm* in the table) are signed (two's complement) numbers, either 8 bits or 16 bits wide, depending on the instruction.

Program Specifications

Create C code for a single binary executable file named "asm".

The asm program should take a single run-time parameter that specifies the path and file name of a file which contains an instructions written in man readable FOOTv8 assembler. The input file must have a file type of ".s", the standard UNIX file type for man-readable assembly code.

The assembled output should be written to a file with the same path and name as the assembly input file, but with the file type changed to ".o". So, for example, if the program were invoked using `./asm pgms/c2f.s`, then the object code output should be written to `pgms/c2f.o`.

To make things simple, the output object file should be an ASCII man readable file. Each instruction from the man-readable assembler should be represented by two 8-digit hexadecimal numbers. The first number represents the offset of the instruction from the beginning of the program. The second represents the binary instruction. For example, if the first instruction is `MOV X0,#4000`, then the first line of output in the object file should be:

00000000 11000fa0

In this case, the offset is 0, the op-code is 0x11 for MOV, the Rd specification is 0x00 for register X0, and the last four digits, 0x0fa0, represent the number 4000 in hexadecimal.

You may write debug information to standard error or standard output. Only the .o file will be checked when grading. It might be useful to produce a “compile listing” output on stdout to help debug your code.

If you find any errors in the input file, such as an unrecognized op-code mnemonic or an immediate value that doesn't fit in the binary immediate field, write an error message to standard output that describes the error and the input assembly file line number, and replace the bad instruction with a NOP instruction in the object file.

Working the Project

The proj1.tar.gz file is available for download from Brightspace. Download and untar this file to create an initial **proj1** directory. The files in this directory are designed to work in the remote.cs.binghamton.edu environment. They will probably work in other UNIX environments as well, but make sure you test on the remote machines.

Design and code an assembler by writing code in the code sub-directory, and updating the code/Makefile to build the executable file, “./asm”. You may add new code files in the code sub-directory, but if you do so, update the Makefile in the code directory so that your code builds correctly.

Test and debug your code from the proj1 directory by running **make test** to run a simple test, **make gdb** to run a simple test under gdb, or **make checkMem** to run a simple test under valgrind to check memory usage. You may want to modify the Makefile to modify the input assembler code used in the simple test. Run **make fullTest** to run all assembly programs provided to you. The **fullTest** target will also run a diff between your output and the correct output if the correct output has been provided to you. (This is the same diff that will be run when grading your code.)

When you are ready to submit, run “make submit” in the proj1 directory. This will create a proj1_code.tar.gz file in the proj1 directory that contains the contents of your code sub-directory. Upload this tar file in the submission area on Brightspace.

Warning – Brightspace prohibits the presence of executable files when you upload. If you get errors uploading your tar file from Brightspace, you may have left an executable file in the code sub-directory.

You may submit as many times as you want before the project deadline. Only the last submission will be graded.

Grading Criteria

Your `proj1_code.tar.gz` file will be downloaded and unzipped on a `remote.cs` server. We will then build your `asm` executable. If the compiler produces error messages while building your code, we will attempt to fix the problem. After your code compiles, it will be run on the `remote.cs` servers using three different `FOOTv8` programs, one of which has been provided to you; the other two are `FOOTv8` assembler programs you have not seen..

By default, you start with 100 points for the project. The following deductions from the maximum grade are possible:

- 15 For each fix required to get an error-free compile, up to a maximum of four fixes.
- 10 If there are any warning messages from the compiler for your code, running with the `-Wall` compiler option. Warning messages often indicate potential bugs; it's worth fixing them!
- 10 If there are output formatting errors in the object file.
- 5 For each incorrect result in the object files produced by your program, up to a maximum of -25 per test case.
- 100 If your code matches or is very similar to another student's code, or code available on the internet. This is an academic honesty violation.
- 100 If you make no attempt to solve the problem.